



Python Programming Fundamentals

Test-Driven Development

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. What is TDD?
2. Unit Testing vs Integration Testing
3. What Makes a Good Test?
4. The BankAccount Class
5. Red-Green-Refactor Cycle
6. Unit vs Integration Test Example



Outline

1. What is TDD?
2. Unit Testing vs Integration Testing
3. What Makes a Good Test?
4. The BankAccount Class
5. Red-Green-Refactor Cycle
6. Unit vs Integration Test Example



1.1 Test-Driven Development

TDD is a development practice where you write **tests before code**:

1. **Red** — Write a failing test (it fails because the code doesn't exist yet)
2. **Green** — Write the **minimum** code to make the test pass
3. **Refactor** — Clean up the code while keeping tests green

Benefits:

- Forces you to think about the interface before implementation
- Every line of code is covered by at least one test
- Bugs are caught immediately
- Tests serve as living documentation



Outline

1. What is TDD?
- 2. Unit Testing vs Integration Testing**
3. What Makes a Good Test?
4. The BankAccount Class
5. Red-Green-Refactor Cycle
6. Unit vs Integration Test Example



2. Unit Testing vs Integration Testing

Unit Test	Integration Test
Tests one class/function in isolation	Tests how components work together
Fast — milliseconds	Slower — may involve files/DB/network
Many, covering each method	Fewer, covering key workflows
Easy to locate failures	Harder to pinpoint failures

We will focus on **unit tests** using pytest.



Outline

1. What is TDD?
2. Unit Testing vs Integration Testing
- 3. What Makes a Good Test?**
4. The BankAccount Class
5. Red-Green-Refactor Cycle
6. Unit vs Integration Test Example



3. What Makes a Good Test?

- **Fast** — should run in milliseconds
- **Isolated** — doesn't depend on other tests or external state
- **Repeatable** — same result every run
- **Self-checking** — uses assert, not manual inspection
- **Named clearly** — `test_deposit_increases_balance()`

A test has three parts (Arrange-Act-Assert / AAA):

Arrange: set up the object/data

Act: call the method

Assert: check the result



Outline

1. What is TDD?
2. Unit Testing vs Integration Testing
3. What Makes a Good Test?
- 4. The BankAccount Class**
5. Red-Green-Refactor Cycle
6. Unit vs Integration Test Example



4. The BankAccount Class

```
class BankAccount:
    def __init__(self, owner: str, balance: float = 0.0):
        self.__owner = owner
        self.__balance = balance

    def deposit(self, amount: float):
        if amount <= 0:
            raise ValueError("Deposit must be positive")
        self.__balance += amount

    def withdraw(self, amount: float):
        if amount <= 0:
            raise ValueError("Withdrawal must be positive")
        if amount > self.__balance:
            raise ValueError("Insufficient funds")
        self.__balance -= amount
```



4. The BankAccount Class

```
def get_balance(self) -> float:  
    return self.__balance
```



Outline

1. What is TDD?
2. Unit Testing vs Integration Testing
3. What Makes a Good Test?
4. The BankAccount Class
- 5. Red-Green-Refactor Cycle**
6. Unit vs Integration Test Example



5. Red-Green-Refactor Cycle

Red: Write a failing test

```
def test_deposit_increases_balance():  
    account = BankAccount("Alice", 100.0)  
    account.deposit(50.0)  
    assert account.get_balance() == 150.0
```

Run it → FAIL (BankAccount doesn't exist yet)

Green: Implement BankAccount.deposit() → test passes

Refactor: Improve the code (rename variables, extract helpers) → tests still green



Outline

1. What is TDD?
2. Unit Testing vs Integration Testing
3. What Makes a Good Test?
4. The BankAccount Class
5. Red-Green-Refactor Cycle
- 6. Unit vs Integration Test Example**



6. Unit vs Integration Test Example

Unit test – tests BankAccount in isolation

```
def test_withdraw_reduces_balance():  
    account = BankAccount("Bob", 200.0)  
    account.withdraw(75.0)  
    assert account.get_balance() == 125.0
```

Integration test – tests Shop save+load together

```
def test_shop_persists_to_json(tmp_path):  
    shop = Shop("Test")  
    shop.add_product(Product("Apple", 1.99, 101))  
    shop.save_to_json(str(tmp_path / "shop.json"))  
  
    shop2 = Shop("Loaded")  
    shop2.load_from_json(str(tmp_path / "shop.json"))  
    assert len(shop2.get_all_products()) == 1
```



Thanks for Watching - Any questions?

